SVD / PCA - All Exam Templates

Combined revision sheet based on the SVD/PCA patterns we identified from the January 2025, January 2026 and June 2026 exam material. Use the common beginning, then choose the ending that matches the wording of the question.

0. Fast story-to-method map

Question wording	What to do
first left/right singular vector	U[:,0] / V[:,0]
retain 90%, 95%, 99% variance	cumulative S**2 -> choose smallest k
first two principal components	project centered X onto V[:,:2]
representative point / class mean	training class centroid
closest class	Euclidean distances -> argmin
classification accuracy	mean(predictions == y_test)
best rank-k / low-rank approximation	truncated SVD reconstruction
poorly represented / anomaly	large reconstruction error
single worst observation	argmax(errors)
top N / exactly N largest errors	argsort(errors)[-N:]
empirical distribution of errors	ECDF
threshold with exactly N above it	midpoint around Nth-largest boundary
random 2D projection	random matrix projection, then compare

1. Master SVD/PCA beginning

Important: Center X when the question asks for PCA/centering. If the question explicitly asks for SVD of the original matrix, use the original X.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# LOAD
df = pd.read_csv("data.csv")

# If there is a target/label:
X = df.drop(columns=["target"]).to_numpy(dtype=float)
y = df["target"].to_numpy()

# If there is NO target/label:
# X = pd.read_csv("data.csv", header=None).to_numpy(dtype=float)

# CENTER - if asked / for PCA
X_mean = np.mean(X, axis=0)
X_centered = X - X_mean
X_svd = X_centered

# If asked for SVD of original X:
# X_svd = X

# COMPACT SVD
U, S, Vt = np.linalg.svd(X_svd, full_matrices=False)

D = np.diag(S)
V = Vt.T

# FIRST SINGULAR VECTORS
first_left = U[:, 0]
first_right = V[:, 0]

# CUMULATIVE EXPLAINED VARIANCE
cumulative_variance = np.cumsum(S**2) / np.sum(S**2)

# CHOOSE SMALLEST k
```

```
THRESHOLD = 0.90   # change to 0.95, 0.99, etc.
k = np.argmax(cumulative_variance >= THRESHOLD) + 1

print("k =", k)
print("variance retained =", cumulative_variance[k - 1])
```

2. PCA projection + 2D plot

Triggers: "project onto first two principal components", "2D PCA representation", "visualize using PC1 and PC2".

```
scores_2d = X_centered @ V[:, :2]

plt.scatter(
    scores_2d[:, 0],
    scores_2d[:, 1],
    c=y
)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

If asked why classes overlap: PCA maximizes retained variance; it does not directly optimize separation between class labels.

3. PCA -> nearest-centroid classification (June-style)

Triggers: "use first k PCs", "first 80% training", "representative/mean point for each class", "closest Euclidean distance", "accuracy".

```
# PCA representation using first k PCs
scores_k = X_centered @ V[:, :k]

# 80/20 split
split = int(0.8 * len(scores_k))

X_train = scores_k[:split]
X_test  = scores_k[split:]
y_train = y[:split]
y_test  = y[split:]

# Training class centroids
classes = np.unique(y_train)
centroids = []

for c in classes:
    centroid = X_train[y_train == c].mean(axis=0)
    centroids.append(centroid)

centroids = np.array(centroids)

# Nearest-centroid prediction
predictions = []

for x in X_test:
    distances = np.linalg.norm(centroids - x, axis=1)
    closest = np.argmin(distances)
    predictions.append(classes[closest])

predictions = np.array(predictions)

# Accuracy
accuracy = np.mean(predictions == y_test)
print("Accuracy:", accuracy)
```

Memory: PCA -> split -> TRAIN centroids -> Euclidean distance -> argmin -> accuracy.

4. Rank-k reconstruction

Triggers: "best rank-k approximation", "low-rank approximation", "reconstruct using first k components".

```
X_reconstructed = (
    U[:, :k]
    @ np.diag(S[:k])
    @ Vt[:k, :]
)
```

5. Reconstruction error / anomaly detection

Triggers: "reconstruction error", "least well represented", "unusual observations", "anomalies/outliers".

```
# Reconstruct
X_reconstructed = (
    U[:, :k]
    @ np.diag(S[:k])
    @ Vt[:k, :]
)

# Euclidean reconstruction error per observation
errors = np.linalg.norm(
    X_svd - X_reconstructed,
    axis=1
)

# ONE worst observation
largest_idx = np.argmax(errors)

# TOP N worst observations
N = 75
top_N = np.argsort(errors)[-N:][::-1]

# Original rows if needed
outlier_rows = df.iloc[top_N]
```

Memory: bad reconstruction = large error = more anomalous.

6. ECDF of reconstruction errors (January 2025-style)

Triggers: "empirical cumulative distribution function", "ECDF of reconstruction errors".

```
sorted_errors = np.sort(errors)

ecdf = (
    np.arange(1, len(sorted_errors) + 1)
    / len(sorted_errors)
)

plt.step(
    sorted_errors,
    ecdf,
    where="post"
)

plt.xlabel("Reconstruction error")
plt.ylabel("ECDF")
plt.show()
```

7. Threshold so exactly N observations are above it

Trigger: "choose a threshold so exactly 10/N samples are classified as outliers."

```
N = 10

sorted_errors = np.sort(errors)

threshold = (
    sorted_errors[-N]
    + sorted_errors[-N - 1]
) / 2

outlier_indices = np.where(
    errors > threshold
)[0]

print("threshold =", threshold)
print("number of outliers =", len(outlier_indices))
```

The midpoint is chosen between the Nth-largest and the next error, so the N largest errors lie above the threshold (assuming no problematic tie at the boundary).

8. First singular vectors only

```
# First LEFT singular vector
u1 = U[:, 0]

# First RIGHT singular vector
v1 = V[:, 0]

# Do NOT use U[0, :] for the first singular vector.
# U[0, :] is the first ROW.
```

9. Random projection comparison - backup pattern

Lower priority, but useful as a backup because an older exam pattern compared PCA in 2D with a random 2D projection.

```
np.random.seed(1)

R = np.random.randn(
    X.shape[1],
    2
)

X_random_2d = X_centered @ R

# Then apply the classifier/prediction procedure
# requested by the question and compare with PCA.
```

Explanation: PCA deliberately selects directions that retain large variation in the data. A random projection does not choose directions based on the data variance, so predictive/classification performance can differ.

10. Full January-2025-style anomaly template

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load original matrix
X = pd.read_csv(
    "SVD.csv",
    header=None
).to_numpy(dtype=float)

# 2. Compact SVD
U, S, Vt = np.linalg.svd(
    X,
    full_matrices=False
)

V = Vt.T

# 3. First singular vectors
first_left = U[:, 0]
first_right = V[:, 0]

# 4. Cumulative explained variance
cum_var = np.cumsum(S**2) / np.sum(S**2)

# 5. Smallest k retaining at least 95%
k = np.argmax(cum_var >= 0.95) + 1

# 6. Rank-k approximation
X_rec = (
    U[:, :k]
    @ np.diag(S[:k])
    @ Vt[:k, :]
)

# 7. Reconstruction errors
errors = np.linalg.norm(
    X - X_rec,
    axis=1
)

# 8. ECDF
sorted_errors = np.sort(errors)
ecdf = (
    np.arange(1, len(sorted_errors) + 1)
    / len(sorted_errors)
)

plt.step(sorted_errors, ecdf, where="post")
plt.xlabel("Reconstruction error")
plt.ylabel("ECDF")
plt.show()

# 9. Threshold for exactly 10 outliers
N = 10

threshold = (
    sorted_errors[-N]
    + sorted_errors[-N - 1]
) / 2

# 10. Select outliers
outlier_indices = np.where(
    errors > threshold
)[0]

outliers = X[outlier_indices]

print("k =", k)
print("threshold =", threshold)
print("number of outliers =", len(outlier_indices))
```

11. Full June-style PCA classification template

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load
```

```
df = pd.read_csv("data.csv")
X = df.drop(columns=["label"]).to_numpy(dtype=float)
y = df["label"].to_numpy()

# 2. Center
mean_X = np.mean(X, axis=0)
Xc = X - mean_X

# 3. Compact SVD
U, S, Vt = np.linalg.svd(
    Xc,
    full_matrices=False
)

V = Vt.T

# 4. Explained variance and k
cum_var = np.cumsum(S**2) / np.sum(S**2)

THRESHOLD = 0.90
k = np.argmax(cum_var >= THRESHOLD) + 1

# 5. First two PCs for visualization
Z2 = Xc @ V[:, :2]

plt.scatter(Z2[:, 0], Z2[:, 1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()

# 6. k-dimensional PCA representation
Z = Xc @ V[:, :k]

# 7. First 80% train, last 20% test
split = int(0.8 * len(Z))

Z_train = Z[:split]
Z_test = Z[split:]
y_train = y[:split]
y_test = y[split:]

# 8. Training centroids
classes = np.unique(y_train)

centroids = np.array([
    Z_train[y_train == c].mean(axis=0)
    for c in classes
])

# 9. Predict nearest centroid
predictions = []

for z in Z_test:
    distances = np.linalg.norm(
        centroids - z,
        axis=1
    )
    predictions.append(
        classes[np.argmin(distances)]
    )

predictions = np.array(predictions)

# 10. Accuracy
accuracy = np.mean(predictions == y_test)

print("k =", k)
print("accuracy =", accuracy)
```

12. Last-minute rules

• Do not automatically center: center when the question/PCA setup asks for it.

• Singular vectors are columns: U[:,0] and V[:,0].

• Explained variance uses squared singular values: S**2.

• Smallest k means find the first cumulative variance value reaching the requested threshold.

• PCA coordinates use centered X multiplied by columns of V.

- Class centroids must use training observations when the question says training/fitting sample.
- Nearest centroid means smallest Euclidean distance -> argmin.
- Anomalies from low-rank reconstruction have LARGE reconstruction errors.
- argmax gives one worst observation; argsort is needed for top N.
- Read the final subquestion: it tells you which SVD branch to use.